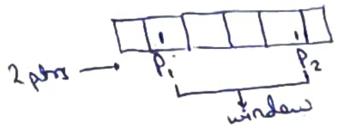
①

Sliding Window - Array

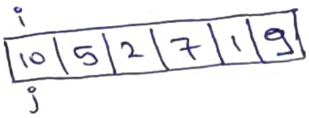
Rules →

- * Contiguous subarray/substring with (+)ve elements in array ('0' can be included)
- * Target 'K' would be given.
- * Type → fixed size
  - Dynamic window → Growing & Shrinking.
- * Basically use 2 pointer to create a window & 1 ptr iterate in for loop to expand
- * Keep 1 ptr fixed to shrink in while loop



Leet-560 Subarray Counts with Sum equals 'K'

- Prefix Sum → in this Qs (-)ve also included
- Sliding window → if only (+)ve & '0' would be there
  T-O(N) S-O(1)



K=15

* i & j = 0
  - → will be iterated regularly (expand window towards right)
  - → j++ only when sum gets more than 'K' (used to shrink window from left)

fixed ptr to use for shrink in while loop

```
int sum=0
int count=0
int j=0
for (i=0 to n)
    sum += num[i];
  while (sum > K && j<=i)
        sum -= num[i];
        j++;
  if (sum == K && j<= i)
        Count ++
  return Count;
```

sum += num[i]; → expanding window toward right by regularly iterating 'i'

while (sum > K && j<=i) → shrink window until sum get smaller or equal to 'K'

sum -= num[i]; → shrinking sum window from left by removing j^th element from left

expanding ptr in for loop

i=0 S=10 j=0
i=1 S = 10+5 = 15 Count ++
i=2 S = 10+5+2 = 17 > K
So, S - arr[i] = S-10 = 7 j=1
i=3 S = 7+1 = 8
i=4 S = 7+1+9 = 17 > K j=
S - arr[i] = S-5 = 12
5+2+1+9
j

2nd Variation of Same above Qs

↳ if need to find longest subarray → 'length' not 'Count'

Sliding window code Same as above

```
int sum=0
int maxlength=0
int j=0
for (i=0 to n)
    sum += arr[i];
while (sum > K && j<=i)
    sum -= arr[j];
      j++;
```

```
if (sum==K && j<=i)
    maxlen = max(maxlen, i-j+1);
Return mxlen;
```

↳ only this part will be changed

Ajanta PRODUCTS Officer

Leet-713 Subarray Product less than 'K'

Range of no (elements of array) → 1 <= num[i] <= 1000
(all positive so sliding window)

1st Cond^n → if (k <= 1) return 0; → there Can't be any number whose product will less than 1 & Here, we have been asked that product 'p' should be strictly less than 'k' so, k should be greater than 1 as least value of 'p' ~~would~~ would be '1' & p ≠ k.

```
int p = 1;
int Count = 0;

int j = 0;              → shrinking pointer (Shrink window from left side)
for (i = 0 to n)        → regular iterating expansion ptr. (expand toward right)
    p *= num[i];        → expand window by including ith elements

    while (p >= k && j <= i)
        p = p/num[j];   → shrink window by removing jth element
        j++

    Count += (i-j+1);   → this is used when we need to find:—
                          * Sum or Product ≤ k
                          * Subarray with at most k distinct element

return Count;             Count ++ → used if Sum or Product exactly equal to 'K' need to be found.
```

Leet-152 (max product Subarray) → ✱ 'K' not given, ✱ also (-)ve no included → So Can't used sliding window

(miscellenious)

Code:—

(jb-jb (-)ve element aayega max wala min aur min wala max ban jayega) →

```
if (num[i] < 0)
    swap(mxP, minP);
```

← Condition

```
maxP = max(num[i], maxP × num[i])
minP = min[num[i], minP × num[i])
ans = max (maxP, ans);
```

{ Striver Hard level }

|1,-1,2,-3| 0 |1,2,3,4| 0 |1,-1,2,3|

Leet-209 Minimum Size Subarray Sum

```
int Sum = 0
int minL = INT_MAX;

int j = 0
for (i = 0 to n)
    Sum += num[i]
    while (Sum >= target)
        minL = min (minL, (i-j+1));    → check while shrinking itself
        Sum -= num[j]
        j++
if (minL == INT_MAX) return 0;          → if no pair (sub array) found
return minL;
```

Leet-Locked AlgoMaster-340 — Longest Substring with at Most 'K' distinct character

↳ Check String Sliding window Notes

Max Consecutive - I — Leet-485

```
int maxC = INT_MIN;
int Count = 0;

for(i=0 to n)
    if(num[i] ==1)
        {Count++}
    else
        { Count=0}
    maxC = max(maxC, Count);
return maxC;
```

Max Consecutive - II — AlgoMaster-487

↳ at most one '0' Can be flipped (K=1)

Max Consecutive - III — LeetCode - 1004

↳ at most K '0's Can be flipped

```
int l=0
    zero=0
    ans=0
for(r=0 to n)              ——→ move (iterate r) regularly
    if(num[r]=0)           ——→ track zeroes, if n[r]=0 then zero++
        zero++
    while(zero > k)        ——→ if no. of zeroes exceed 'k' then r loop paused & while loop will
                               Start iterating l
        if(num[l]=0)   }
            zero--     }——→ iterate 'l' & as it passes zeroes n[l]=0 reduce zero--
        l++            }       So, that again for loop for 'r' Could be resumed.
    ans = max(ans, r-l+1);
return ans;
```

'if' also Can be used here (← while)

T= O(2N) : Here it's not nested loop, for every r we are not iterating l but instead we are passing r when zero exceed 'k' then moving 'l' to reduce zeroes

S-O(1)

Fruit Basket-I :—→ Dynamic Sliding Window + HASHMap O(n)

Fruit Basket-II :—→ Segment Tree (Binary Tree) O(n log n)

Fruit Basket-III —→ Segment Tree (Binary Tree) O(n log n)

↳ First-fit placement problem

Leet-904 Fruit into Basket

T – O(n) S → (O(1))

→ Donot use set it will fail if duplicate elements arrives Consecutively {Check leet code wrong marked solutions}

window size = 2 (fixed)
Hash map is used as window

```
int l = 0
for (r = 0 to n)
    m[fruits[r]]++;
    while (m.size() > 2)
        m[fruits[l]]--;
        if (m[fruits[l]] == 0)
            m.erase(fruits[l]);
        l++
    Count = max(Count, r-l+1);
```

m[fruits[r]]++; → exceeding window

Shrink windows always using while loop.

m[fruits[l]]--; → removing each duplicate one by one

if (m[fruits[l]] == 0) → when no duplicate left remove the key from ~~elem~~map
else map - {3:0, 1:1, 2:1}
↓
even '0' occurence remains stored in map

m.erase(fruits[l]); → so, erase key itself

l++ → move l (shrink)

Leet-2461 Max Sum of Distinct Subarray with length K

→ Same as above Q., 3 ways to solve → check on leet-Code

→ T – O(n) S – O(K) or O(1)

→ 2 Conditions
- → No duplicates
- → Size of subarray should be 'K' only

→ So use 2 shrink window 1st → for duplicates
then add element in map & sum if it has not already occured
then 2nd shrink for keep size = 'K'.

```
int j=0
for (i=0 to n)
    while (Seen.count(num[i]))
        Seen[num[i]]--
        Sum -= num[i]
        if (Seen[num[i]]==0)
            Seen.erase(num[i])
        j++
    Seen[num[i]]++
    Sum +=Sum[i];
    while (i-j+1 > K)
    if (i-j+1 ==K) maxC = max(maxC, Sum);
Return maxC;
```

1st shrink for duplicates

→ check explanation in above Qs.
↓
this can be modified check leet code 3 solution are submitted there.

add into window only if it's not duplicate

2nd shrink for size of sum array

→ update maxC only if size of subarray is 'K'

②

Sliding Window Array

Leet-1838 Freq of Most freq Element

Approaches →

- i) Brute force → T: $O(n^2)$ S: $O(1)$
  - ↳ check leet
- ii) Prefix + Binary → T: $O(n \log n)$ S: $O(n)$
  - ↓ check binary.S notes
- most optimal → iii) Sliding Window → T: $O(n \log n)$ S: $O(1)$ or $O(n)$

Sliding Window

```
Sort(num.begin(), num.end())

long CurrentSum = 0
int  ans = 0

int j=0
for(i=0 to n)
    long target = nums[i]

    Currentsum += target

while(target *(i-j+1)-CurrSum > k)
   CurrentSum -= num[j]
   j++;

ans = max(ans, i-j+1);

return ans;
```

→ Sort the array first

[1, 8, 13, 4]

⇓

[1, 4, 8, 13]

j → i

→ Current element as target.

→ we will keep each element as target & make elements of left side equal to target

→ [1, 4, ⑧, 13]

↳ i=2, target = nums[2] = 8

Curr Sum = 1+4+8 = $\boxed{13}$

→ if we make all elements as 8 Sum will be target × (i-j+1)

target × (2-0+1)

target × 3 = 8×3 = $\boxed{24}$

Required Cost = 24 − 13 = 11

But K = 5

So, K < req Cost

Shrink the window from left

Improvised as advance Sliding

Check leetcode.